

TECHNICAL WALKTHROUGH – TIME TRACKING PIPELINE

VEERASAK NICHAYAPUN



AGENDA

- Objective
- Features
- Architecture
- Considerations and thought processes
- Future designs on periodic ingestion
- Data Validation strategies

OBJECTIVE

- Identify issues in the dataset and plan the data transformation and cleaning processes using ETL methods.
- Establish a reliable and scalable data pipeline to streamline data collection, transformation, and loading processes.
- Enable data visualization on the web app for tracking team members and project time in table form.
- Provide a flexible framework that can easily adapt to new data sources and changing business requirements.

FEATURES

User Check-ins

Select a user:

ned

The following is a list of check-ins for ned

	user	project	hours	timestamp
0	ned	bizdev	8	2019-09-27 00:00:00.000000
1	ned	bizdev	4	2019-09-26 00:00:00.000000
2	ned	cultureandmanagement	1	2019-09-26 00:00:00.000000
3	ned	project-00	1.5	2019-09-26 00:00:00.000000
4	ned	project-43	1	2019-09-26 00:00:00.000000
5	ned	project-66	1	2018-11-26 12:08:09.035700
6	ned	cultureandmanagement	1.5	2018-11-26 12:08:09.035700
7	ned	learning	0.5	2018-11-26 12:08:09.035700
8	ned	events	2.5	2018-11-26 12:08:09.035700
9	ned	cultureandmanagement	0.5	2018-11-26 12:08:09.035700
10	ned	transit	1.5	2018-11-26 12:08:09.035700
11	ned	project-66	1	2018-11-23 10:22:27.010200
12	ned	project-28	1	2018-11-23 10:22:27.010200

The web service allows you to carry out the following functions:

- Choose from the list of available users
- Review all check-ins along with their corresponding hours and timestamps
- Sort project names, hours, and timestamps in either ascending or descending order.

ARCHITECTURE

Data Source:

A CSV file with 4 columns and 20,500 rows

Extract Layer:

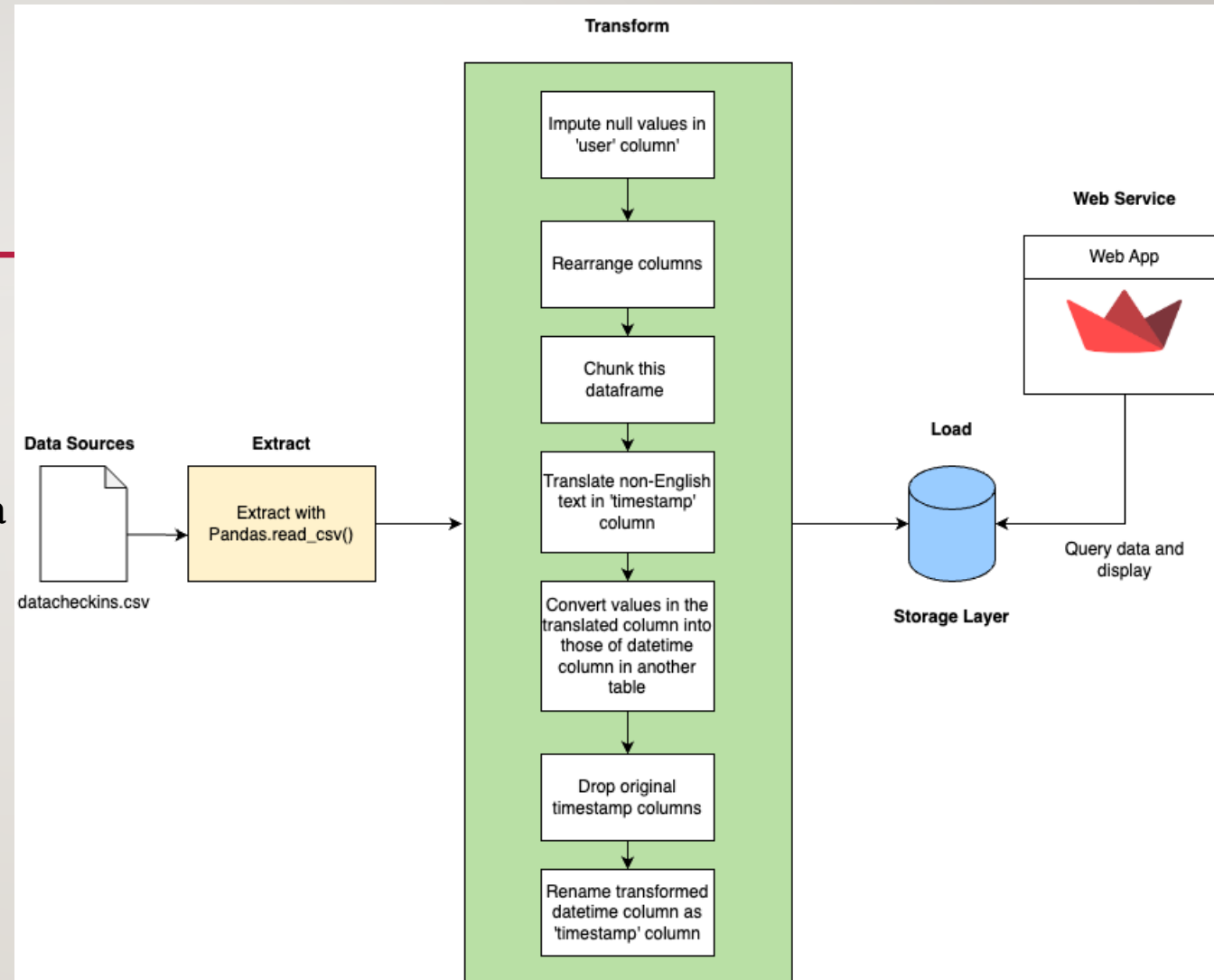
Pandas.read_csv() to convert a csv file into a data frame.

Transform Layer:

A combination of imputation, translation, Data type conversion and column rearrangement

Load Layer:

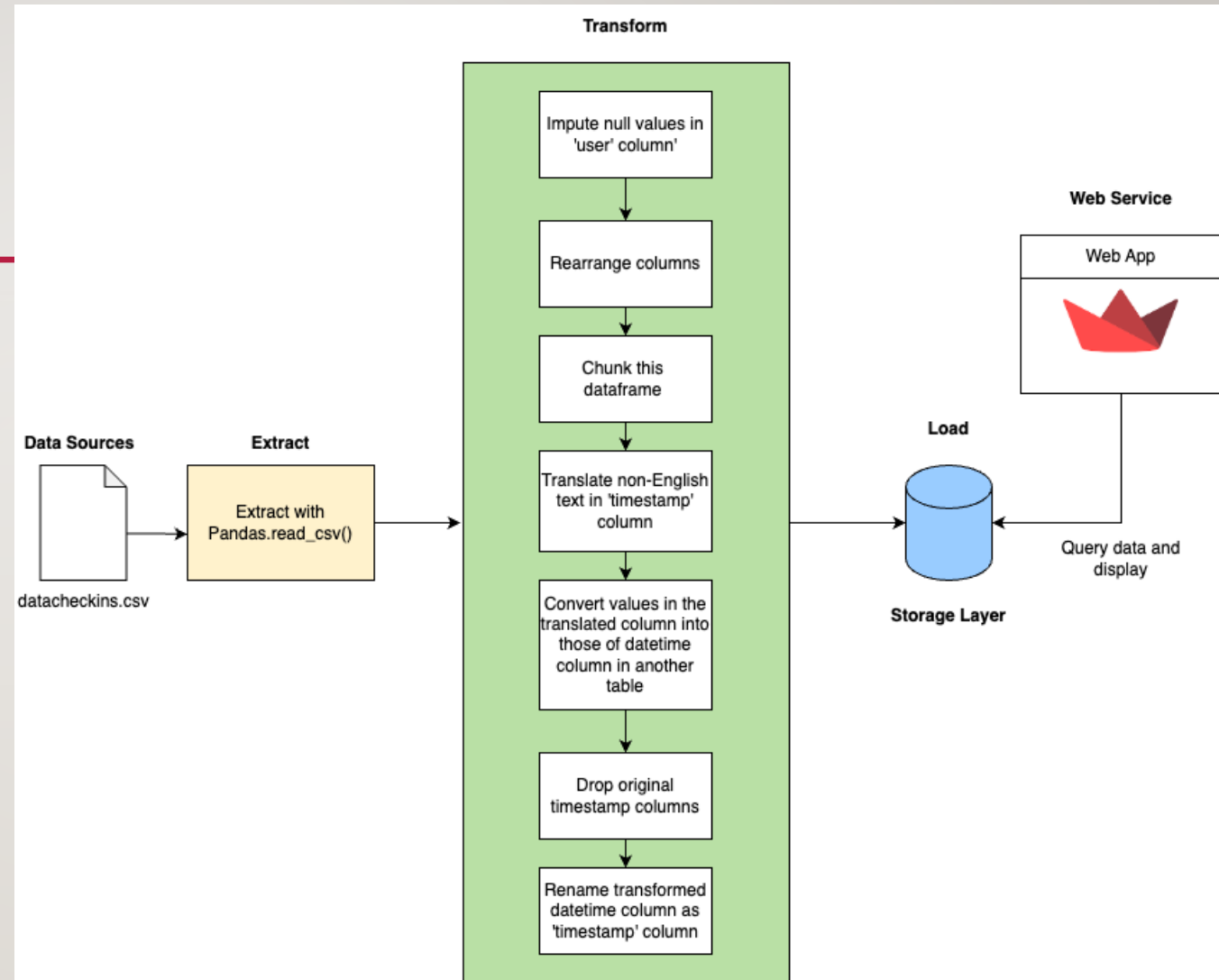
Pandas.to_sql() to load the cleaned data to the table in a sqlite database



ARCHITECTURE (CONT.)

Web Service:

- Streamlit-based web app
- Provide an instant deployment on the web app using Python code



URL: <https://poomer-tm-time-tracking-main-q6plwv.streamlit.app/>

TECH STACKS : 4 LAYERS OF PYTHON LIBRARIES

Data Processing Layer

Pandas: The powerful python framework for data manipulation, cleaning and analysis. This is useful for ETL processes implementation.

Language Translation Layer

langdetect: Language detection library to detect non-English text.

deep_translator: A framework to translate between different languages.

Database Management Layer

sqlalchemy: The Python SQL toolkit and Object Relational Mapper that enables you to create a database and perform DDL and DML on it.

Web App Layer

streamlit: A python framework that enables you to transform Python scripts into interactive web apps in minutes with instant code deployment.

CHALLENGES

Dataset Challenge	
<u>Issue</u> Null values on 'user' column	<u>Decision</u> Imputation on these null values, replacing them with a default 'unknown user'
<u>Reason:</u> This unknown user could be identified later. Therefore, dropping these rows could result in the loss of data	
<u>Issue</u> Disorganized column orders in the original dataset	<u>Decision</u> Rearrange column orders during transformation phase
<u>Reason:</u> The disorganized column orders make it harder to analyze data.	
<u>Issue</u> Non-English text found in timestamp column	<u>Decision</u> Use Deep_translator to translate them into English text
<u>Reason:</u> These non-English text would obstruct the time-related calculation and need to be urgently fixed	
<u>Issue</u> Diverse datetime format in the form of string type	<u>Decision</u> Use pd.to_datetime() to force convert them into those of datetime type
<u>Reason:</u> The diverse datetime format makes it cumbersome for time-related analysis	
Scalability Challenge	
<u>Issue</u> Translator function fails to translate on a big dataset	<u>Decision</u> Break this massive dataset into a list of smaller batches/chunks
<u>Reason:</u> It is possible that the resource limitation on this function could be a reason. Thus, divide-and-conquer method works.	

IMPLEMENTATION CONSIDERTATIONS

The selection of a technology stack is based on the following key considerations.

Easy integration with Python:

- These stacks are all Python-based, minimizing potential language conversion issues that could increase development time and costs.

Rapid development for this POC:

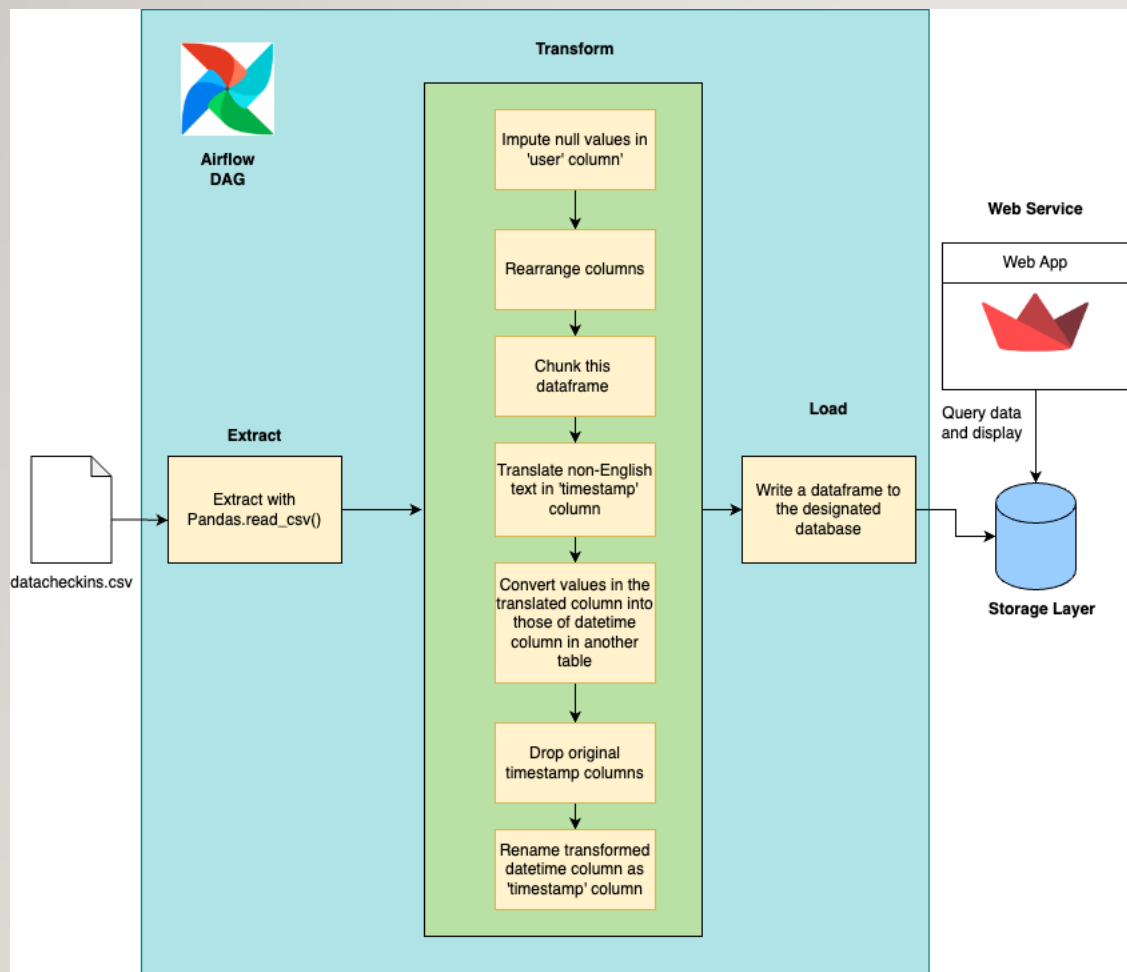
- Streamlit is a web framework that enables quick publication of data with built-in visualizations. Deploying it is simple, requiring just the upload of Python code without infrastructure management. Nevertheless, we may need to switch to RESTful API if the web becomes more complex.

Low operational overhead:

- Sqlite is chosen for its simple database credential management, as security isn't a major concern for this task. However, a security layer should be considered for future POC expansions.



SOLUTION ON PERIODIC INGESTION #1: AIRFLOW



Scheduler : Airflow DAG

A schedule for the ingestion can be configured using 'schedule' parameter of the DAG

Task : Airflow PythonOperator

Convert all Python ETL steps (highlighted in yellow) Into PythonOperator tasks and reorder them under the DAG created above.

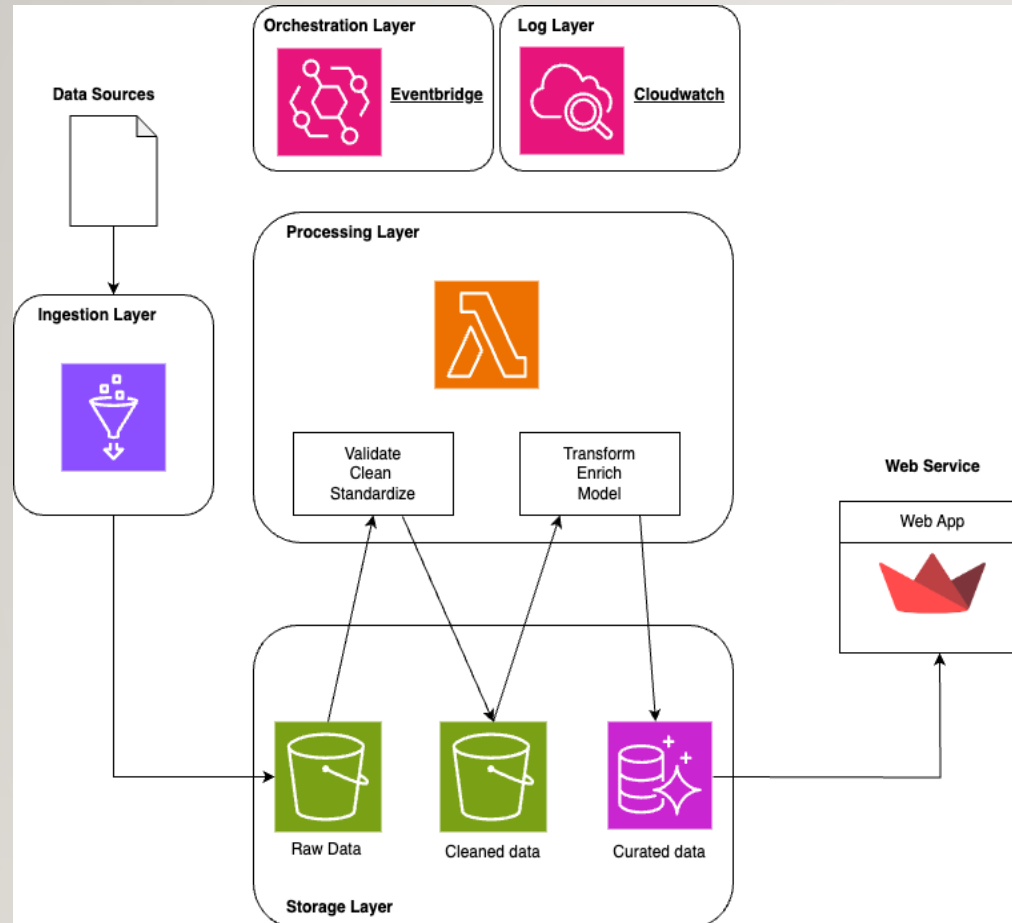
Pros:

- Free and low-cost
- Python-based

Cons:

- Lots of development overhead
- Not scalable

SOLUTION ON PERIODIC INGESTION #2: AWS



Scheduler: Eventbridge

The cron job can be configured and directly issue a lambda function (processing layer)

Storage: The cron job can be configured and directly issues a lambda function (processing layer)

Pros:

- Less code management compared to Airflow
- Scalable and serverless (on all layers shown here)

Cons:

- More expensive due to request fees

DATA VALIDATION STRATEGIES ON INGESTED DATA

Row count validation:

- Since I decide not to remove any null users, the row count of the raw data and transformed data should be identical.

Column validation:

- The column names of the source/raw dataset must be identical to those of the transformed dataset.

Checksum and Hashing:

- Use hash function e.g. MD5 or SHA256 to create a checksum of the raw data and processed data.

Null value and duplicate check:

- Use `pandas.isna()` to check if any null values still exist.
- Use `pandas.duplicated()` to check duplicated values in the ingested data.

THANK YOU

